Εθνικό Μετσόβιο Πολυτεχνείο

Σχολή Ηλεκτρολόγων Μηχανικών & Μηχανικών Υπολογιστών

Εξάμηνο 6ο : Συστήματα Μικροϋπολογιστών

Δημήτριος Θανόγιαννης – 03121902

3η Ομάδα Ασκήσεων

Ασκήσεις προσομοίωσης:

1η Άσκηση:

Ακολουθεί ο κώδικας που υλοποιεί τη ζητούμενη εφαρμογή σε C:

```c
#include <stdint.h>

#define RCC_AHB1ENR  (*(volatile uint32_t *)0x40023830)
#define RCC_APB1ENR  (*(volatile uint32_t *)0x40023840)

#define GPIOA_MODER  (*(volatile uint32_t *)0x40020000)
#define GPIOA_AFRL   (*(volatile uint32_t *)0x40020020)
#define GPIOA_ODR    (*(volatile uint32_t *)0x40020014)

#define GPIOD_MODER  (*(volatile uint32_t *)0x40020C00)
#define GPIOD_IDR    (*(volatile uint32_t *)0x40020C10)

#define USART2_SR    (*(volatile uint32_t *)0x40004400)
#define USART2_DR    (*(volatile uint32_t *)0x40004404)
#define USART2_BRR   (*(volatile uint32_t *)0x40004408)
#define USART2_CR1   (*(volatile uint32_t *)0x4000440C)

#define RCC_APB2ENR    (*(volatile uint32_t *)0x40023844)
#define SYSCFG_EXTICR2 (*(volatile uint32_t *)0x4001380C)
#define EXTI_IMR       (*(volatile uint32_t *)0x40013C00)
#define EXTI_RTSR      (*(volatile uint32_t *)0x40013C08)
#define EXTI_PR        (*(volatile uint32_t *)0x40013C14)
#define NVIC_ISER0     (*(volatile uint32_t *)0xE000E100)

const char covid_seq[] =
    "AUUAAAGGUUUAUACCUUCCCAGGUAACAAACCAACCAACUUUCGAUCUCUUGUAGAUCU"
    "GUUCUCUAAACGAACUUUAAAAUCUGUGUGGCUGUCACUCGGCUGCAUGCUUAGUGCACU"
    "CACGCAGUAUAAUUAAUAACUAAUUACUGUCGUUGACAGGACACGAGUAACUCGUCUAUC"
    "UUCUGCAGGCUGCUUACGGUUUCGUCCGUGUUGCAGCCGAUCAUCAGCACAUCUAGGUUU"
    "CGUCCGGGUGUGACCGAAAGGUAAGAUGGAGAGCCUUGUCCCUGGUUUCAACGAGAAAAC"
    "ACACGUCCAACUCAGUUUGCCUGUUUUACAGGUUCGCGACGUGCUCGUACGUGGCUUUGG"
    "AGACUCCGUGGAGGAGGUCUUAUCAGAGGCACGUCAACAUCUUAAAGAUGGCACUUGUGG"
    "CUUAGUAGAAGUUGAAAAAGGCGUUUUGCCUCAACUUGAACAGCCCUAUGUGUUCAUCAA"
    "ACGUUCGGAUGCUCGAACUGCACCUCAUGGUCAUGUUAUGGUUGAGCUGGUAGCAGAACU"
    "CGAAGGCAUUCAGUACGGUCGUAGUGGUGAGACACUUGGUGUCCUUGUCCCUCAUGUGGG"
    "CGAAAUACCAGUGGCUUACCGCAAGGUUCUUCUUCGUAAGAACGGUAAUAAAGGAGCUGG"
```

```c
void delay_ms(uint32_t ms) {
    for(uint32_t i = 0; i < ms * 4000; i++) {   // assuming 16 MHz clock and 4 cycles per loop, we have 4000 iterations per ms
        __asm("NOP");                           // insert no operations for delay simulation
    }
}

void uart_putc(char c) {
    while (!(USART2_SR & (1<<7)));  // pause while UART is busy sending a character (Transmit Data Register Empty = 0)
    USART2_DR = c;                  // when it becomes available again, put c in Data Register
}

void uart_puts(const char *str) {           // function to print an entire string
    while(*str) uart_putc(*str++);          // while c != '\0' (end of string), send c to UART
}

void uart_putn(uint32_t num) {          // function to appropriately depict numbers
    char buf[10];
    int i = 0;
    if (num == 0) { uart_putc('0'); return; }   // 0 is depicted as '0' in ASCII, no change
    while (num > 0) { buf[i++] = (num % 10) + '0'; num /= 10; } // translate all digits of a number to its correct ASCII value, using '0' as positio
    while (i > 0) { uart_putc(buf[--i]); }      // print them in the correct (reverse) order
}

char get_complement(char c) {       // RNA complement calculation
    if(c == 'A') return 'U';
    if(c == 'U') return 'A';
    if(c == 'G') return 'C';
    if(c == 'C') return 'G';
    return 0;
}

volatile int reset_search_flag = 0; // global search reset flag
```

```c
void EXTI4_IRQHandler(void) {
    if (EXTI_PR & (1<<4)) {     // if EXTI4
        EXTI_PR |= (1<<4);      // clear pending bit

        GPIOA_ODR |= (1<<13);   // turn on PA13
        delay_ms(3000);         // for 3 seconds
        GPIOA_ODR &= ~(1<<13);  // and then turn it off

        reset_search_flag = 1;  // set flag = 1 for the app_main to see and reset
    }
}

int app_main(void) {
    RCC_AHB1ENR |= (1<<0) | (1<<3); // enable clock for GPIOA, GPIOD
    RCC_APB1ENR |= (1<<17);         // enable clock for USART2

    GPIOA_MODER &= ~(0x0F0F00F0);
    GPIOA_MODER |= 0x050A00A0;      // set GPIOA PA12, PA13 as outputs, PA2 as TX, PA3 as RX (alternate function)
    GPIOA_AFRL  |= 0x00007700;      // set alternate function 7 for the desired functionality USART2

    GPIOD_MODER &= ~(3 << 8);       // set GPIOD PD4 as input

    RCC_APB2ENR |= (1<<14);             // enable SYSCFG clock

    SYSCFG_EXTICR2 &= ~(0x0000000F);    // clear bits 0-3 (EXTI4)
    SYSCFG_EXTICR2 |= 0x00000003;       // set EXTI4 in Port D (0011)

    EXTI_IMR |= (1<<4);                 // enable line 4
    EXTI_RTSR |= (1<<4);                // rising edge enable

    NVIC_ISER0 |= (1<<10);              // enable EXTI4 interrupt in NVIC (position 10)

    USART2_BRR = 0x008B;                // set appropriate Baud Rate for 16 MHz clock
    USART2_CR1 = (1<<13) | (1<<3) | (1<<2); // enable USART enable bit, Transmitter enable bit, Receiver enable bit
```

```
    char input[100];
    char comp[100];

    while(1) {
        int idx = 0;
        int reset_search = 0;

        uart_puts("Insert RNA sequence: ");

        while(1) {
            if (reset_search_flag) {    // look for interrupt
                reset_search_flag = 0;  // clear flag
                reset_search = 1;
                uart_puts("\r\n[!] Search Reset by PD4 !\r\n\n");
                break;
            }

            if (USART2_SR & (1<<5)) {   // check for new character in RX
                char c = USART2_DR;

                if (c == '\n' || c == '\r') {
                    if (idx > 0) break; // end of sequence
                } else if (c == 'A' || c == 'U' || c == 'G' || c == 'C') {
                    input[idx++] = c;
                    uart_putc(c);           // if it is accepted, print it
                }
            }
        }

        if (reset_search) continue;     // if there was an interrupt (PD4), do not print the accumulated characters

        input[idx] = '\0';              // finish the sequence
        uart_puts("\r\n");                  // change line

         for(int i = 0; i < idx; i++) {
             comp[i] = get_complement(input[i]); // calculate complementary sequence
         }
         comp[idx] = '\0';                      // finish the sequence

         uart_puts("Complement to find: ");
         uart_puts(comp);
         uart_puts("\r\nPositions: ");

         int count = 0;
         int seq_len = sizeof(covid_seq) - 1;

         for(int i = 0; i <= seq_len - idx; i++) {   // find matches in Covid-19 sequence
             int match = 1;
             for(int j = 0; j < idx; j++) {
                 if (covid_seq[i+j] != comp[j]) {
                     match = 0; break;
                 }
             }
             if (match) {
                 count++;
                 uart_putn(i);                          // keep all of the starting positions, if there are any
                 uart_puts(" ");
             }
         }

         uart_puts("\r\nTotal occurrences: ");
         uart_putn(count);                              // print them
         uart_puts("\r\n");

         if (count > 0) {                           // if there is at least an occurrence
             GPIOA_ODR |= (1<<12);                  // turn on PA12
             delay_ms(2000);                        // for 2 seconds
             GPIOA_ODR &= ~(1<<12);                 // and then close it
         }
    }
}
```

και σε assembly:

```
.syntax unified
.thumb
.text
.global main_app
.type main_app, %function

.equ RCC_BASE,    0x40023800
.equ RCC_AHB1ENR, 0x30
.equ RCC_APB1ENR, 0x40

.equ GPIOA_BASE,  0x40020000
.equ GPIOD_BASE,  0x40020C00
.equ MODER_OFFSET, 0x00
.equ AFRL_OFFSET,  0x20
.equ ODR_OFFSET,   0x14
.equ IDR_OFFSET,   0x10

.equ USART2_BASE, 0x40004400
.equ USART_SR,    0x00
.equ USART_DR,    0x04
.equ USART_BRR,   0x08
.equ USART_CR1,   0x0C

delay_ms:
    LDR R1, =4000           // assuming 16 MHz clock and 4 cycles per loop, we have 4000 iterations per ms
    MUL R0, R0, R1          // R0 is the function argument
    CMP R0, #0
    BEQ delay_end
delay_loop:
    NOP                     // insert no operations for delay simulation
    SUBS R0, R0, #1
    BNE delay_loop
delay_end:
    BX LR

```

```
uart_putc:
    LDR R1, =USART2_BASE        // R0 is the function argument
putc_wait:
    LDR R2, [R1, #USART_SR]
    ANDS R2, R2, #(1<<7)
    BEQ putc_wait               // pause while UART is busy sending a character (Transmit Data Register Empty = 0)
    STRB R0, [R1, #USART_DR]    // when it becomes available again, put c in Data Register
    BX LR

uart_puts:                      // function to print an entire string
    PUSH {R4, LR}               // keep R4 and LR in stack because they will change values
    MOV R4, R0                  // R0 is the function argument
puts_loop:
    LDRB R0, [R4], #1           // Byte-wise (character length)
    CMP R0, #0                  // while c != '\0' (end of string)
    BEQ puts_end
    BL uart_putc                // send c to UART
    B puts_loop
puts_end:
    POP {R4, PC}                // return R4 and put LR to PC

uart_putn:                      // function to appropriately depict numbers
    PUSH {R4-R7, LR}            // keep R4-R7 and LR in stack because they will change values
    MOV R4, R0                  // R0 is the function argument
    CMP R4, #0
    BNE putn_calc
    MOV R0, #'0'
    BL uart_putc                // 0 is depicted as '0' in ASCII, no change
    POP {R4-R7, PC}             // return R4-R7 and put LR to PC
putn_calc:
    LDR R5, =10
    MOV R6, #0                  // length counter
    SUB SP, SP, #12             // temporary buffer in stack
    MOV R7, SP
```

```
putn_loop:
    CMP R4, #0
    BEQ putn_print
    UDIV R0, R4, R5             // R0 = R4 / 10
    MLS R1, R0, R5, R4          // R1 = R4 - (R0 * 10) (modulo equation)
    ADD R1, R1, #'0'            // translate all digits of a number to its correct ASCII value, using '0' as position reference
    STRB R1, [R7, R6]
    ADD R6, R6, #1
    MOV R4, R0
    B putn_loop
putn_print:
    CMP R6, #0
    BEQ putn_end            // if length = 0 skip
    SUB R6, R6, #1
    LDRB R0, [R7, R6]
    BL uart_putc            // print them in the correct (reverse) order
    B putn_print
putn_end:
    ADD SP, SP, #12         // empty temporary buffer in stack
    POP {R4-R7, PC}         // return R4-R7 and put LR to PC

```

```
get_complement:             // RNA complement calculation, R0 is the function argument
    CMP R0, #'A'
    BNE 1f
    MOV R0, #'U'
    BX LR
1:  CMP R0, #'U'
    BNE 2f
    MOV R0, #'A'
    BX LR
2:  CMP R0, #'G'
    BNE 3f
    MOV R0, #'C'
    BX LR
3:  CMP R0, #'C'
    BNE 4f
    MOV R0, #'G'
    BX LR
4:  MOV R0, #0
    BX LR

```

```
main_app:
    PUSH {R4-R11, LR}   // keep R4-R11 and LR in stack

    LDR R0, =RCC_BASE
    LDR R1, [R0, #RCC_AHB1ENR]
    ORR R1, R1, #((1<<0) | (1<<3))
    STR R1, [R0, #RCC_AHB1ENR] // enable clock for GPIOA, GPIOD

    LDR R1, [R0, #RCC_APB1ENR]
    ORR R1, R1, #(1<<17)
    STR R1, [R0, #RCC_APB1ENR] // enable clock for USART2

    LDR R0, =GPIOA_BASE
    LDR R1, [R0, #MODER_OFFSET]
    LDR R2, =0x0F0F00F0
    BIC R1, R1, R2
    LDR R2, =0x050A00A0
    ORR R1, R1, R2
    STR R1, [R0, #MODER_OFFSET] // set GPIOA PA12, PA13 as outputs, PA2 as TX, PA3 as RX (alternate function)

    LDR R1, [R0, #AFRL_OFFSET]
    LDR R2, =0x00007700
    ORR R1, R1, R2
    STR R1, [R0, #AFRL_OFFSET] // set alternate function 7 for the desired functionality USART2

    LDR R0, =GPIOD_BASE
    LDR R1, [R0, #MODER_OFFSET]
    LDR R2, =(3 << 8)
    BIC R1, R1, R2
    STR R1, [R0, #MODER_OFFSET] // set GPIOD PD4 as input

    LDR R0, =USART2_BASE
    LDR R1, =0x008B
    STR R1, [R0, #USART_BRR] // set appropriate Baud Rate for 16 MHz clock

     LDR R1, =((1<<13) | (1<<3) | (1<<2))
     STR R1, [R0, #USART_CR1] // enable USART enable bit, Transmitter enable bit, Receiver enable bit

// Register Map:
// R4: Input sequence length (idx)
// R5: Search reset flag (1 if PD4 pressed) / Match counter
// R6: Universal iterator (i) / COVID sequence length
// R7: Outer loop iterator (search index)
// R8: Inner loop iterator (pattern match index)
// R9: Match boolean flag (1 if matched, 0 if failed)

main_loop:
    MOV R4, #0              // idx = 0
    MOV R5, #0              // reset_search = 0

    LDR R0, =msg_insert
    BL uart_puts

input_loop:                 // PD4 polling instead of iterrupt handling
    LDR R0, =GPIOD_BASE
    LDR R1, [R0, #IDR_OFFSET]
    ANDS R1, R1, #(1<<4)
    BEQ check_uart          // if PD4 is on

    LDR R0, =GPIOA_BASE
    LDR R1, [R0, #ODR_OFFSET]
    ORR R1, R1, #(1<<13)
    STR R1, [R0, #ODR_OFFSET] // turn on PA13

    LDR R0, =3000
    BL delay_ms             // for 3 seconds

```

```
    LDR R0, =GPIOA_BASE
    LDR R1, [R0, #ODR_OFFSET]
    BIC R1, R1, #(1<<13)
    STR R1, [R0, #ODR_OFFSET] // and then turn it off

wait_pd4_release:             // wait until it is not pressed (avoid multiple interrupts)
    LDR R0, =GPIOD_BASE
    LDR R1, [R0, #IDR_OFFSET]
    ANDS R1, R1, #(1<<4)
    BNE wait_pd4_release      // do not move on until it is not pressed anymore

    MOV R5, #1              // reset_search = 1
    LDR R0, =msg_reset
    BL uart_puts
    B end_input_loop

check_uart:
    LDR R0, =USART2_BASE
    LDR R1, [R0, #USART_SR]
    ANDS R1, R1, #(1<<5)
    BEQ input_loop          // check for new character in RX

    LDR R0, =USART2_BASE
    LDRB R6, [R0, #USART_DR]

    CMP R6, #'\n'
    BEQ handle_enter
    CMP R6, #'\r'
    BEQ handle_enter
    B check_nucleotides

handle_enter:
    CMP R4, #0
    BGT end_input_loop      // end of sequence
    B input_loop

check_nucleotides:
    CMP R6, #'A'
    BEQ accept_char
    CMP R6, #'U'
    BEQ accept_char
    CMP R6, #'G'
    BEQ accept_char
    CMP R6, #'C'
    BEQ accept_char
    B input_loop

accept_char:
    LDR R0, =input_buf
    STRB R6, [R0, R4]       // R4 is the length counter
    ADD R4, R4, #1
    MOV R0, R6
    BL uart_putc            // if it is accepted, print it
    B input_loop

end_input_loop:
    CMP R5, #1
    BEQ main_loop           // if PD4 was turned on (reset_flag = 1), do not print the accumulated characters

    LDR R0, =input_buf
    MOV R1, #0
    STRB R1, [R0, R4]       // finish the sequence

    LDR R0, =msg_newline
    BL uart_puts            // change line

    MOV R6, #0              // i = 0 for complement loop
```

```
comp_loop:
    CMP R6, R4
    BGE end_comp_loop
    LDR R0, =input_buf
    LDRB R0, [R0, R6]
    BL get_complement       // calculate complementary sequence
    LDR R1, =comp_buf
    STRB R0, [R1, R6]
    ADD R6, R6, #1
    B comp_loop
end_comp_loop:
    LDR R0, =comp_buf
    MOV R1, #0
    STRB R1, [R0, R4]       // finish the sequence by inserting null terminator ('\0')

    LDR R0, =msg_comp
    BL uart_puts
    LDR R0, =comp_buf
    BL uart_puts
    LDR R0, =msg_pos
    BL uart_puts

    MOV R5, #0              // count = 0
    MOV R6, #0              // sequence length
strlen_loop:
    LDR R0, =covid_seq
    LDRB R1, [R0, R6]
    CMP R1, #0
    BEQ do_search
    ADD R6, R6, #1
    B strlen_loop

do_search:
    MOV R7, #0              // i = 0
search_outer:
    SUB R1, R6, R4          // sequence length - idx
    CMP R7, R1
    BGT search_end          // find matches in Covid-19 sequence

    MOV R9, #1              // match = 1
    MOV R8, #0              // j = 0
search_inner:
    CMP R8, R4
    BGE inner_end
    LDR R0, =covid_seq
    ADD R1, R7, R8          // i + j
    LDRB R1, [R0, R1]
    LDR R2, =comp_buf
    LDRB R2, [R2, R8]
    CMP R1, R2
    BEQ next_j
    MOV R9, #0
    B inner_end
next_j:
    ADD R8, R8, #1
    B search_inner
inner_end:
    CMP R9, #1
    BNE next_i
    ADD R5, R5, #1
    MOV R0, R7
    BL uart_putn            // keep all of the starting positions, if there are any
    LDR R0, =msg_space
    BL uart_puts
next_i:
    ADD R7, R7, #1
    B search_outer
```

```
search_end:
    LDR R0, =msg_total
    BL uart_puts
    MOV R0, R5
    BL uart_putn            // print them
    LDR R0, =msg_newline
    BL uart_puts

    CMP R5, #0
    BLE main_loop           // if there is at least an occurrence

    LDR R0, =GPIOA_BASE
    LDR R1, [R0, #ODR_OFFSET]
    ORR R1, R1, #(1<<12)
    STR R1, [R0, #ODR_OFFSET] // turn on PA12

    LDR R0, =2000
    BL delay_ms               // for 2 seconds

    LDR R0, =GPIOA_BASE
    LDR R1, [R0, #ODR_OFFSET]
    BIC R1, R1, #(1<<12)
    STR R1, [R0, #ODR_OFFSET] // and then close it

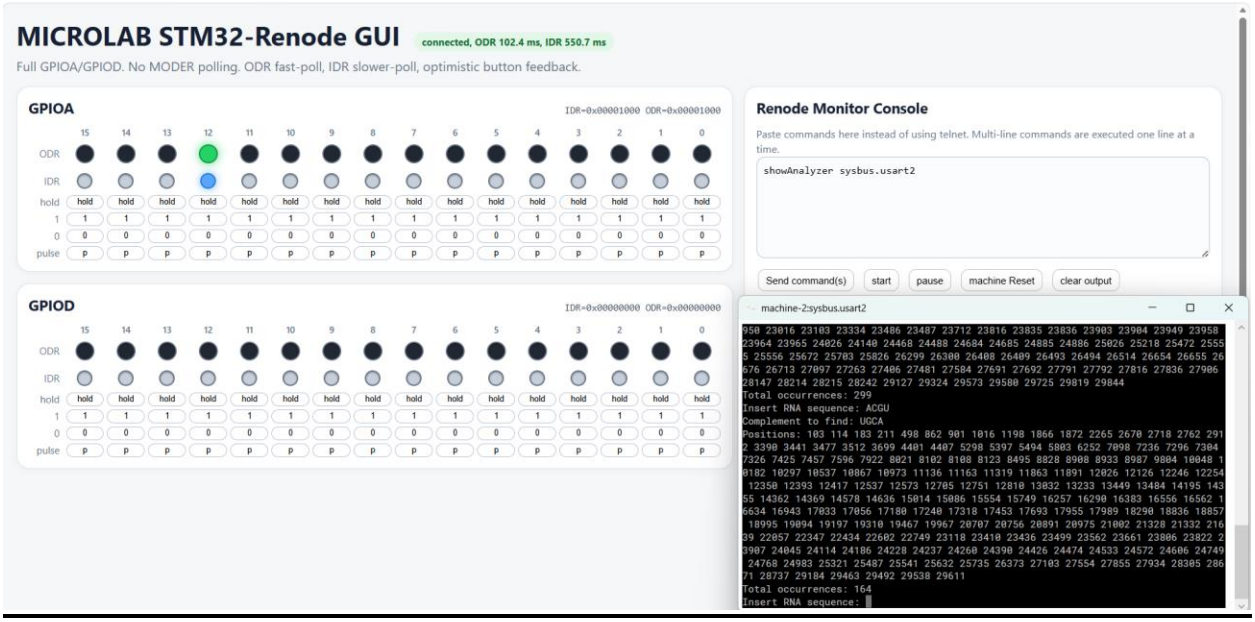
    B main_loop

.align 2
msg_insert:  .asciz "Insert RNA sequence: "
msg_reset:   .asciz "\r\n[!] Search Reset by PD4 !\r\n\n"
msg_newline: .asciz "\r\n"
msg_comp:    .asciz "Complement to find: "
msg_pos:     .asciz "\r\nPositions: "
msg_space:   .asciz " "
msg_total:   .asciz "\r\nTotal occurrences: "

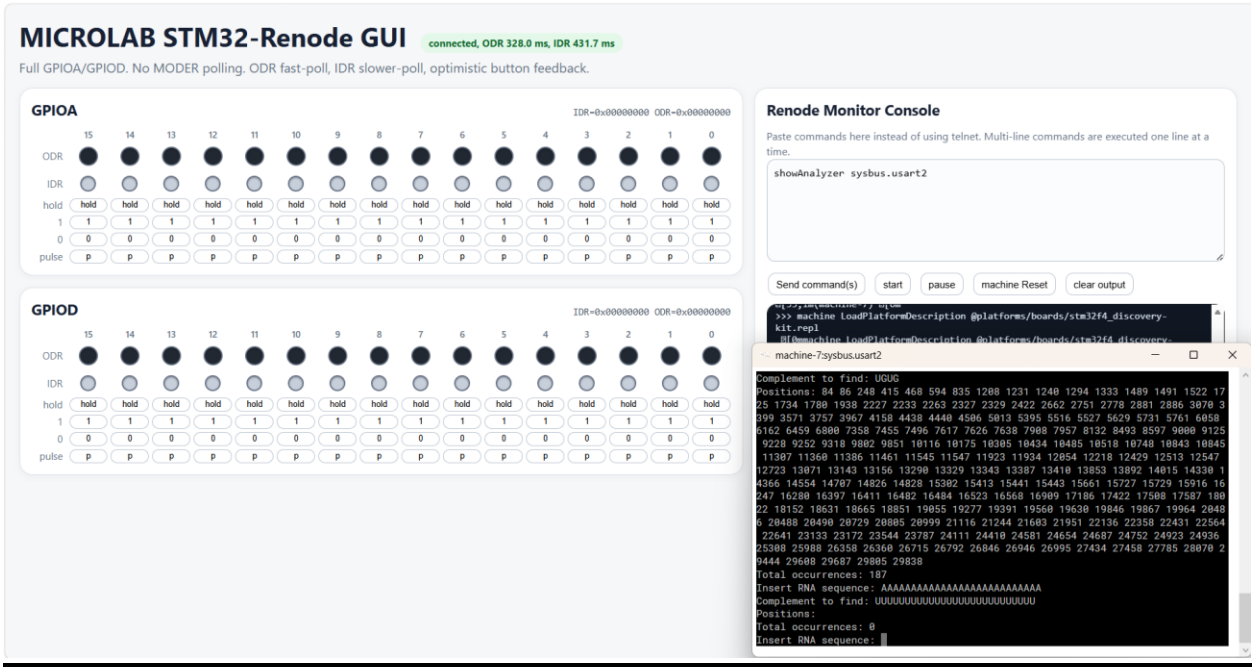
.bss
.align 2
input_buf: .space 100
comp_buf:  .space 100

.data
.align 2
covid_seq:
    .asciz "AUUAAAGGUUUAUACCUUCCCAGGUAACAAACCAACCAACUUUCGAUCUCUUGUAGAUCUGUUCUCUAAACGAACUUUAAAAUCUGUGUGGCUGUCACUCGGCUGCAUGCUUAGUGCACUCACGCAGUAUAAUUAAU
```

Παρακάτω φαίνεται η ορθή λειτουργία του κώδικα (ίδια και στις 2 υλοποιήσεις, με μοναδική διαφορά το χειρισμό EXTI4 interrupt στη C, PD4 polling στην assembly), όπου μόνο οι χαρακτήρες ACUG (case sensitive) γίνονται αποδεκτοί από το σύστημα, το συμπλήρωμα της ακολουθίας υπολογίζεται σωστά, η αναζήτηση γίνεται σωστά και επιστρέφονται όλες οι αρχικές θέσεις εμφάνισης της αλληλουχίας, μαζί με το πλήθος τους. Στην επιστροφή αποτελέσματος, ανάβει το PA12 για 2 δευτερόλεπτα.

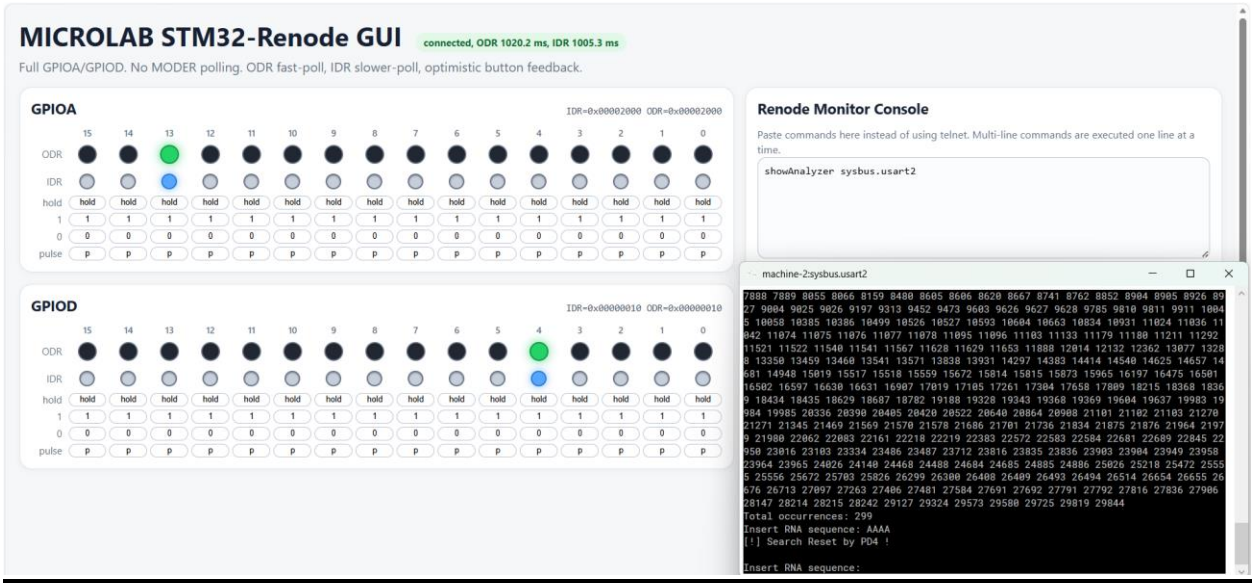


Στην απουσία αντιστοίχισης, εμφανίζεται μηδενικό αποτέλεσμα και δεν ανάβει κανένα LED.



Ενώ κατά τη διάρκεια της πληκτρολόγησης, αν πατηθεί το PD4, η λειτουργία διακόπτεται, η πληκτρολογημένη ακολουθία χάνεται, το σύστημα γίνεται reset και ανάβει το PA13 για 3 δευτερόλεπτα. Όπως ζητείται από την εκφώνηση, για την ανίχνευση του πατήματος του PD4, στην υλοποίηση της C χρησιμοποιήθηκε εξωτερική διακοπή (EXTI4 interrupt), ενώ στην

αντίστοιχη υλοποίηση της assembly η ανίχνευση έγινε μέσω συνεχούς ελέγχου (polling) της κατάστασης του καταχωρητή.



Θεωρητικές Ασκήσεις:

2η Άσκηση:

α) Η μικρότερη χωρητικότητα των Flash Memory και SRAM που μπορεί να υποστηρίξει το SoC μας χωρίς να υπάρχει σφάλμα στην εκτέλεση, είναι:

Flash Memory: περιέχει τα section .text και section .rodata, καθώς είναι read only memory. Όμως, επειδή η SRAM είναι volatile, τα δεδομένα της πρέπει να αποθηκεύονται σε read only μορφή και στη flash, ενώ στην αρχή εκτέλεσης του προγράμματος, δημιουργείται αντίγραφό τους στην SRAM, ώστε να μπορούν να γίνουν και τροποποιήσεις τους. Τα δεδομένα της External RAM δε θα αποθηκευτούν στη flash, καθώς δεν είναι απαραίτητα για την ομαλή εκκίνηση του προγράμματος (είναι δυναμικά δεδομένα που δίνονται κατά το run-time, σε αντίθεση με το compile-time στα δεδομένα της SRAM). Επομένως, η flash θα περιέχει και το section .data, με αποτέλεσμα να απαιτείται ελάχιστη χωρητικότητα 1 KB + 512 MB + 512 MB = **1024,001 MB**, ώστε να μην υπάρξει σφάλμα στην εκτέλεση.

SRAM: περιέχει μόνο το section .data, καθώς είναι read/write memory. Οπότε, απαιτείται ελάχιστη χωρητικότητα **512 MB,** ώστε να μην υπάρξει σφάλμα στην εκτέλεση.

β) Η χαρτογράφηση των address spaces των .text, .rodata, .data regions, καθώς και των GPIO περιφερειακών και της external RAM, βασιζόμενοι στο datasheet του ARM Cortex-M4, είναι η ακόλουθη:
.text: 0x00000000 – 0x000003FF (Flash Memory)

.rodata: 0xA0000000 – 0xBFFFFFFF (External Device (εξωτερική μνήμη flash, καθώς δε χωράει στην εσωτερική μνήμη flash, λόγω της παρουσίας του κώδικα .text))

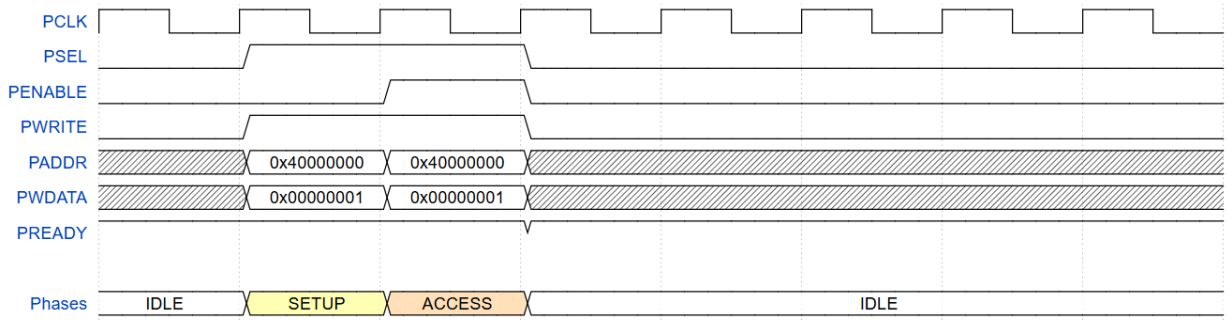
.data: 0x20000000 – 0x3FFFFFFF (SRAM)

GPIO devices: 0x48000000 – 0x48000C13 (GPIOA: 0x48000000 - 0x48000013, GPIOB: 0x48000400 - 0x48000413, GPIOC: 0x48000800 - 0x48000813, GPIOD: 0x48000C00 - 0x48000C13) (Peripheral)

External RAM: 0x60000000 – 0x67FFFFFF (External RAM)

3η Άσκηση:

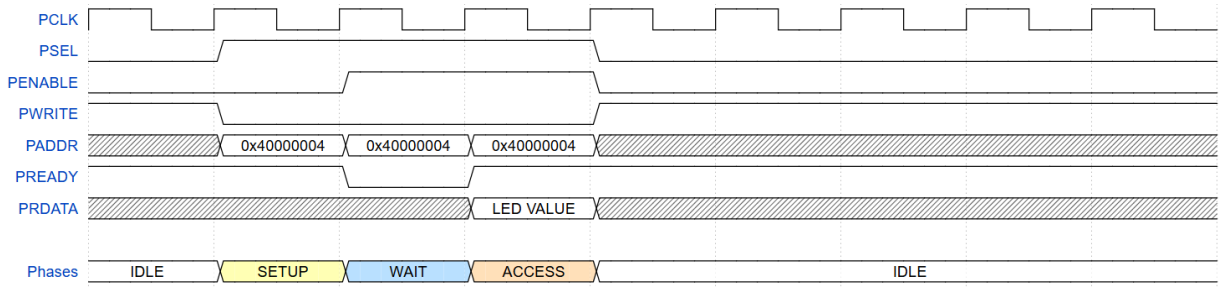
α) Το διάγραμμα χρονισμού για την εγγραφή της τιμής 1 στο LED μέσω του καταχωρητή LED_CTRL είναι το ακόλουθο:



Το APB ξεκινά στον 1ο κύκλο στην κατάσταση IDLE, όπου PSELx = 0 και PENABLE = 0. Στο 2ο κύκλο βρισκόμαστε στην κατάσταση SETUP,όπου το PSELx του περιφερειακού (LED) γίνεται 1, ενώ το PENABLE του παραμένει 0, φάση που διαρκεί 1 κύκλο. Στον ίδιο κύκλο φορτώνονται τα δεδομένα (PADDR, PWDATA), ενώ το PWRITE γίνεται 1 (H) για να σηματοδοτήσει εγγραφή στον καταχωρητή LED_CTRL του περιφερειακού. Στον 3ο κύκλο

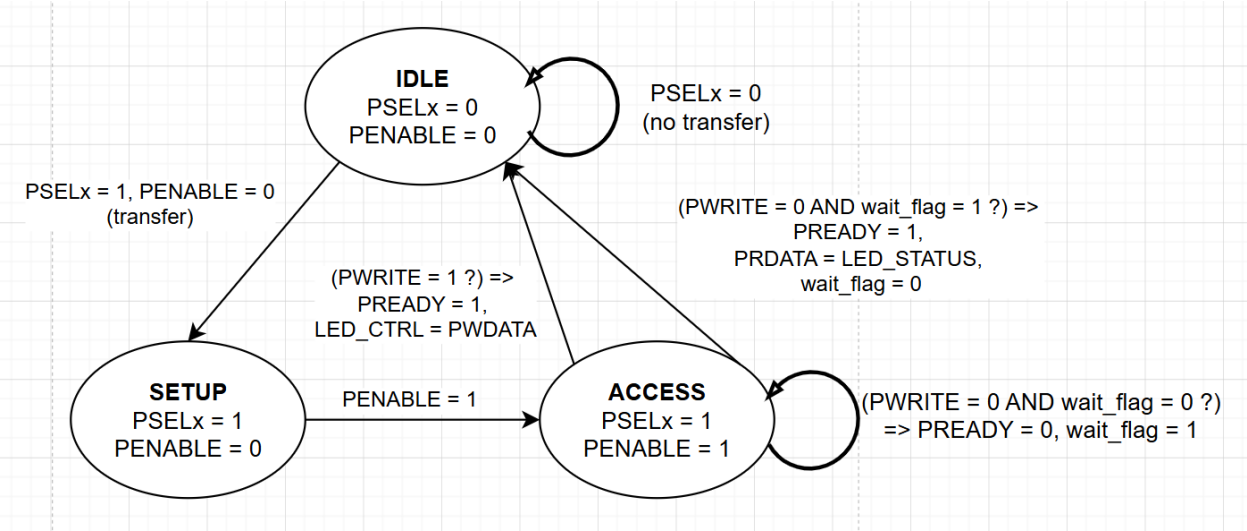
βρισκόμαστε στην κατάσταση ACCESS, όπου και το PENABLE γίνεται 1 και ενεργοποιείται το περιφερειακό για να ολοκληρωθεί η μεταφορά των δεδομένων. Στον 4ο κύκλο (no wait states), το PREADY παραμένει 1 και οι διάδρομοι των δεδομένων αδειάζουν, με αποτέλεσμα το APB να επαναφέρεται στην κατάσταση IDLE, όπου τα σήματα PSELx, PENABLE μηδενίζονται και το PWRITE περνάει σε κατάσταση Don't Care.

Το διάγραμμα χρονισμού για την ανάγνωση της τιμής του LED μέσω του καταχωρητή LED_STATUS είναι το ακόλουθο:



Το APB ξεκινά στον 1ο κύκλο στην κατάσταση IDLE, όπου PSELx = 0 και PENABLE = 0. Στο 2ο κύκλο βρισκόμαστε στην κατάσταση SETUP,όπου το PSELx του περιφερειακού (LED) γίνεται 1, ενώ το PENABLE του παραμένει 0, φάση που διαρκεί 1 κύκλο. Στον ίδιο κύκλο φορτώνεται η διεύθυνση (PADDR), ενώ το PWRITE γίνεται 0 (L) για να σηματοδοτήσει ανάγνωση από τον καταχωρητή LED_STATUS του περιφερειακού. Στον 3ο κύκλο (1 wait state) βρισκόμαστε στην κατάσταση WAIT, όπου και το PENABLE γίνεται 1 και ενεργοποιείται το περιφερειακό για να ξεκινήσει η ανάγνωση των δεδομένων, ενώ ταυτόχρονα το PREADY γίνεται 0 για να αναγκάσει το APB να παραμείνει στην κατάσταση ACCESS. Στον 4ο κύκλο, φορτώνονται τα δεδομένα από τον καταχωρητή LED_STATUS στο διάδρομο και το PREADY γίνεται 1, σηματοδοτώντας τη λήξη του wait state. Στον 5ο κύκλο, εφόσον το PREADY είναι 1 και οι διάδρομοι των δεδομένων αδειάζουν, το APB επαναφέρεται στην κατάσταση IDLE, όπου τα σήματα PSELx, PENABLE μηδενίζονται και το PWRITE περνάει σε κατάσταση Don't Care.

β) Το FSM το όποιο υλοποιεί τον ελεγκτή διεπαφής του APB περιφερειακού, ελέγχει την κατάσταση των εισόδων του, και παράγει με σωστό χρονισμό τα σήματα PREADY, PRDATA βάση της προδιαγραφής του χρονισμού του APB, είναι το ακόλουθο:



Το APB ξεκινά από τη default κατάσταση IDLE παραμένει σε αυτή όσο δε γίνεται κάποια μεταφορά δεδομένων στο περιφερειακό (PSELx = 0). Όταν απαιτείται κάποια μεταφορά δεδομένων, το PSELx γίνεται 1 και το APB μεταβαίνει στην κατάσταση SETUP, στην οποία παραμένει αυστηρά για ένα κύκλο. Στον επόμενο κύκλο, και το σήμα PENABLE γίνεται 1 και το APB μεταβαίνει στην κατάσταση ACCESS. Αν έχουμε εγγραφή στο περιφερειακό (PWRITE = 1), τότε γράφονται τα δεδομένα από το PWDATA στο περιφερειακό, το PREADY γίνεται 1 και εφόσον δεν εκκρεμεί άλλη μεταφορά δεδομένων, το APB μεταφέρεται ξανά στην κατάσταση IDLE. Αν έχουμε ανάγνωση από το περιφερειακό (PWRITE = 0), τότε αν το wait_flag είναι 0 (βοηθητική μεταβλητή για την αναπαράσταση του ενός wait state), το PREADY γίνεται 0 και το wait_flag γίνεται 1. Αν το wait_flag είναι 1, τότε το PREADY γίνεται 1, το PRDATA μεταφέρει τα δεδομένα του καταχωρητή LED_STATUS του περιφερειακού και το wait_flag γίνεται 0. Αμέσως, εφόσον δεν εκκρεμεί άλλη μεταφορά δεδομένων, το APB μεταβαίνει στην κατάσταση IDLE.